El flujo de la información

TP2 — UART + ALU

Del byte que sale de la PC al byte que vuelve,
siguiendo el caminito bloque por bloque

Cómo leer este apunte

- **Verde:** la intuición. La analogía antes de la fórmula.
- **Azul:** la ficha técnica. Señales, números y ciclos exactos.
- **Naranja:** lo que conviene mirar dos veces, o preguntar en la cátedra.
- **Violeta:** la idea central de la sección. Si leés solo esto, ya ganaste algo.

Arquitectura de Computadoras — FCEFyN, UNC
Basys 3 (Artix-7) — clock de 100 MHz — 19200 baud

El TP2 entero en una página

La PC manda tres bytes por un cable: el dato **A**, el dato **B** y el **código de operación**. La placa contesta un byte: el **resultado**. Eso es todo lo que se ve desde afuera.

Adentro pasa lo siguiente. El cable trae *un bit por vez* y no trae reloj: llega un *nivel* que sube y baja, y la FPGA tiene que adivinar dónde empieza y termina cada bit. Para adivinar bien usa un truco: cuenta mucho más rápido que el cable. Genera un pulso llamado `s_tick` **16 veces por cada bit**, y con esos 16 pulsos se ubica justo en el medio del bit, donde la señal está más estable, y ahí la mira.

Una vez que juntó los 8 bits, el byte entra en una cola (`fifo_rx`) y sale del mundo lento del cable para entrar al mundo rápido de los 100 MHz. Ahí lo levanta una máquina de estados (`intf`) que va llenando tres registros: primero A, después B, después OP. Cuando tiene los tres, la ALU — que es combinacional pura, sin reloj — ya tiene el resultado en su salida. La FSM lo mete en otra cola (`fifo_tx`), el transmisor lo saca bit por bit al mismo ritmo de 16 ticks por bit, y el byte vuelve a la PC.

La idea que hay que llevarse: en este sistema conviven **tres velocidades** muy distintas — el clock (10 ns), el tick (3,26 µs) y el bit (52,16 µs) — y *cada bloque vive en una de ellas*. Las colas FIFO existen justamente para que un bloque rápido y uno lento puedan hablarse sin perder datos. Entender quién vive en qué velocidad es entender el TP.

Índice general

El TP2 entero en una página **1**

1 El vocabulario mínimo **3**

2 El mapa: quién le habla a quién **3**

3 Las tres escalas de tiempo **4**

4 El viaje de ida: de la PC a la ALU **5**
4.1 Paso 0 — La PC suelta el byte . . . 5
4.2 Paso 1 — `baud_gen`: el metrónomo que nunca para . . . 5
4.3 Paso 2 — El sincronizador: dos flip-flops antes de tocar nada . . . 6
4.4 Paso 3 — `uart_rx`: contar ticks para encontrar el centro . . . 6
4.5 Paso 4 — `rx_done_tick`: el aviso de un ciclo . . . 8
4.6 Paso 5 — `fifo_rx`: la frontera entre los dos mundos . . . 8
4.7 Paso 6 — `intf`: armar la operación . . . 9
4.8 Paso 7 — La ALU: el bloque sin reloj . . . 9

5 El viaje de vuelta: de la ALU a la PC **10**
5.1 Paso 8 — `SEND` y `fifo_tx` . . . 10
5.2 Paso 9 — `uart_tx` arranca solo . . . 10
5.3 Paso 10 — El byte sale bit por bit . . . 10

6 El cronograma completo **11**

7 Tabla integradora: todas las señales de un vistazo **12**

8 Las siete preguntas que más se hacen **12**

9 Autoevaluación **13**

1 El vocabulario mínimo

Seis palabras. Si estas seis están claras, el resto del apunte se lee solo.

Las seis palabras

Baud rate. Bits por segundo que viajan por el cable. Acá: 19200. Es la velocidad del *cable*, no la del chip.

Trama (frame). El paquetito completo que viaja por el cable: 1 bit de start (siempre 0) + 8 bits de datos + 1 bit de stop (siempre 1). 10 bits en total para transportar 8.

Sobremuestreo. Mirar la línea muchas veces por bit. Acá, 16. Sirve para ubicar el centro del bit con precisión de $1/16$ de bit.

Tick (`s_tick`). Un pulso de **un solo ciclo de clock** de ancho, que aparece 16 veces por bit. Es el metrónomo del UART.

Pulso de un ciclo. Una señal que vale 1 durante exactamente un ciclo de clock (`rx_done_tick`, `tx_done_tick`). Sirve para avisar *"pasó algo"* sin trabar nada.

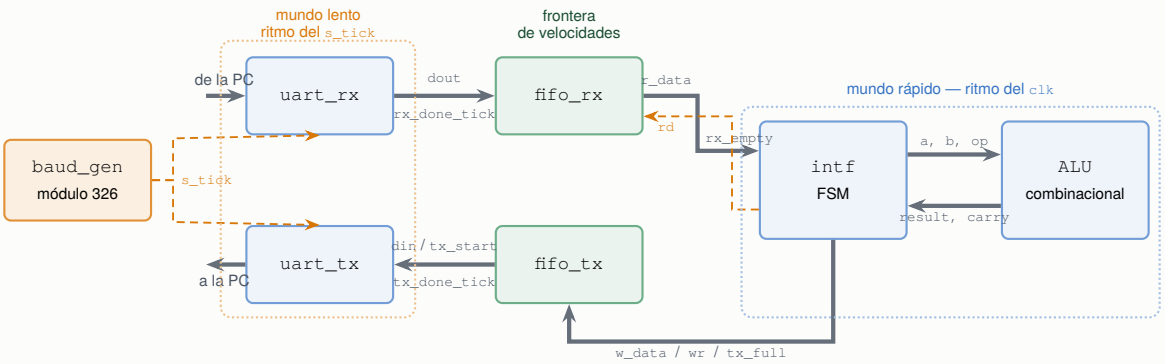
FIFO. Una cola. Se escribe por un lado, se lee por el otro, y los dos lados pueden ir a velocidades distintas.

Por qué "asíncrono"

El cable tiene **una sola línea de datos y ningún reloj**. Es como alguien que te habla desde la otra punta de la cancha sin que ustedes dos hayan acordado un metrónomo: vos sabés *a qué velocidad* habla (los 19200 baud, pactados de antemano), pero no sabés *cuándo* va a empezar la próxima palabra. Por eso cada trama arranca con un bit de start: es el "ey, escuchame" que dispara tu cronometraje.

2 El mapa: quién le habla a quién

Antes de recorrer el camino, la foto completa. Las flechas gruesas son datos (8 bits), las finas son señales de control (1 bit).



El reparto de tareas

Bloque	Qué hace	A qué ritmo vive
`baud_gen`	Cuenta 326 ciclos y tira un tick	clock (100 MHz)
`uart_rx`	Cuenta ticks y arma el byte	tick (306 kHz)
`fifo_rx`	Frontera lento → rápido	los dos lados
`intf`	Ordena A, B, OP y pide el resultado	clock (100 MHz)
`ALU`	Calcula. No tiene reloj.	instantáneo (comb.)
`fifo_tx`	Frontera rápido → lento	los dos lados
`uart_tx`	Cuenta ticks y saca el byte	tick (306 kHz)

3 Las tres escalas de tiempo

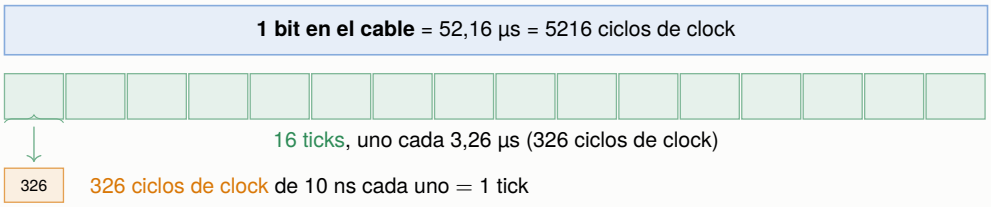
Esta es la sección que hay que entender antes que ninguna otra. Todo el resto del apunte se apoya acá.

La analogía del tren

Pensá en tres relojes encajados uno adentro del otro:

- El **clock** es el segundero de un cronómetro: tic cada 10 ns. Todo lo que es `always @(posedge clk)` se mueve con él.
- El **tick** es una campanita que suena **cada 326 tics** del cronómetro. El `baud_gen` es el único que la hace sonar.
- El **bit** en el cable dura **16 campanitas**.

El clock es el único reloj físico. El tick y el bit son ficciones que construimos contando: el tick contando ciclos de clock, el bit contando ticks.



Los números, de una vez

$$M = \frac{f_{clk}}{\text{BaudRate} \times 16} = \frac{100\,000\,000}{19200 \times 16} = 325{,}52 \approx \mathbf{326}$$

Escala	Período	Frecuencia	Quién la genera
Ciclo de clock	10 ns	100 MHz	el cristal de la placa
Tick (`s_tick`)	3,26 µs	306,7 kHz	`baud_gen` contando 326
Bit en el cable	52,16 µs	19,18 kbaud	`uart_rx`/`uart_tx` contando 16 ticks
Trama de 10 bits	521,6 µs	—	la trama completa

El error de redondeo, y por qué no molesta

326 no es 325,52. El bit real dura 52,16 µs en vez de 52,083: un error de **+0,147 por ciento**.

→ **¿Alcanza?** El receptor se resincroniza en cada bit de start, así que el error solo se acumula durante **una trama**. El último bit que mira (el de stop) lo muestrea 9,5 bits después del flanco de start. Para no equivocarse, el corrimiento acumulado tiene que ser menor a medio bit:

$$\text{tolerancia} = \frac{0{,}5}{9{,}5} = 5{,}26 \text{ por ciento}$$

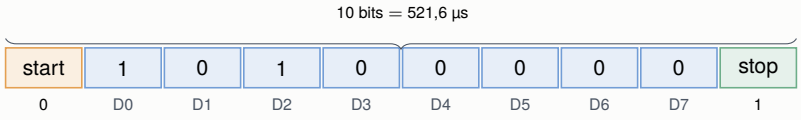
Tenemos 0,147. Estamos 35 veces por debajo del límite. Por eso no hace falta ninguna corrección fraccionaria.

4 El viaje de ida: de la PC a la ALU

Ahora sí, el caminito. Vamos a seguir **un solo byte** — digamos el dato A = `0x05` — desde que sale de la PC hasta que queda guardado en un registro adentro de la FPGA.

4.1 Paso 0 — La PC suelta el byte

La PC pone en el cable, en orden y a 19200 bits por segundo:



El bit menos significativo va primero

`0x05` = `0000 0101` en binario, pero por el cable sale `1010 0000`: **D0 primero**. Es por esto que en `uart_rx` el shift register se arma metiendo el bit nuevo *por izquierda*:

```
b_next = {rx, b_reg[DBIT-1:1]};
```

Cada bit nuevo empuja a los anteriores hacia la derecha. Cuando entró el octavo, el primero que llegó quedó en el bit 0. Si esto se hace al revés el byte sale espejado y es uno de los bugs clásicos del TP.

4.2 Paso 1 — `baud_gen`: el metrónomo que nunca para

El `baud_gen` no espera nada ni le importa lo que pase en el cable. Arranca con el reset y cuenta para siempre: $0, 1, 2, \ldots, 325, 0, 1, \ldots$ Cuando el contador vale 325 pone `tick` en 1 durante **un solo**

ciclo de clock, y en el siguiente vuelve a 0.

La línea que importa

```
assign tick = (count_reg == (M - 1));
```

Es combinacional sobre el valor *actual* del contador, no un registro. Por eso el pulso dura exactamente un ciclo: al ciclo siguiente el contador ya vale 0 y la comparación da falso. Esto es lo que hace que `s_tick` sirva como *habilitación* (*clock enable*) y no como un segundo reloj.

Por qué no se usa el tick como clock

Tentación típica: "si necesito algo que vaya a 306 kHz, ¿por qué no hago `always @(posedge tick)`?". Porque eso crea un **segundo dominio de reloj** dentro del chip, con todos los problemas de cruce de dominios que eso trae, y además en una FPGA una señal generada por lógica no tiene la red de distribución de reloj: llega con *skew* a cada flip-flop.
La forma correcta es la que usa el TP: **todo corre con el mismo clock de 100 MHz**, y el tick se usa como una condición (`if (s_tick) ...`) que habilita el avance. Así hay un solo dominio de reloj en todo el diseño.

4.3 Paso 2 — El sincronizador: dos flip-flops antes de tocar nada

La línea `i_rx` viene de un chip externo y cambia cuando se le canta. No tiene ninguna relación con el clock de 100 MHz. Si la metés directo a un flip-flop y justo cambia en el instante del flanco, el flip-flop puede quedar **metaestable**: ni 0 ni 1, oscilando un rato antes de decidirse.

Los dos flip-flops del top

```
rx_meta <= i_rx;     rx_sync <= rx_meta;
```

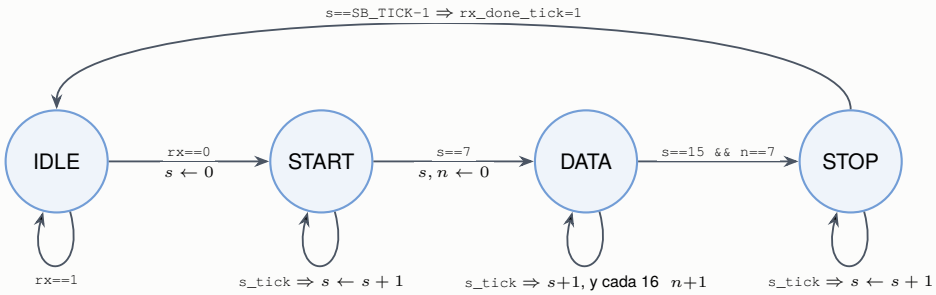
El primero es el que se banca la metaestabilidad; el segundo le da un ciclo entero para que se estabilice antes de que el resto del diseño lo vea. Costo: **20 ns de retardo**. Contra un bit de 52 160 ns, es 0,04 por ciento. Gratis.

En el `.xdc` esto se acompaña con `set_false_path -from [get_ports i_rx]`: le avisa a Vivado que no analice el timing de esa entrada, porque ya sabemos que es asíncrona y ya la tratamos.

4.4 Paso 3 — `uart_rx`: contar ticks para encontrar el centro

Acá está el corazón del TP. La FSM tiene cuatro estados y dos contadores:

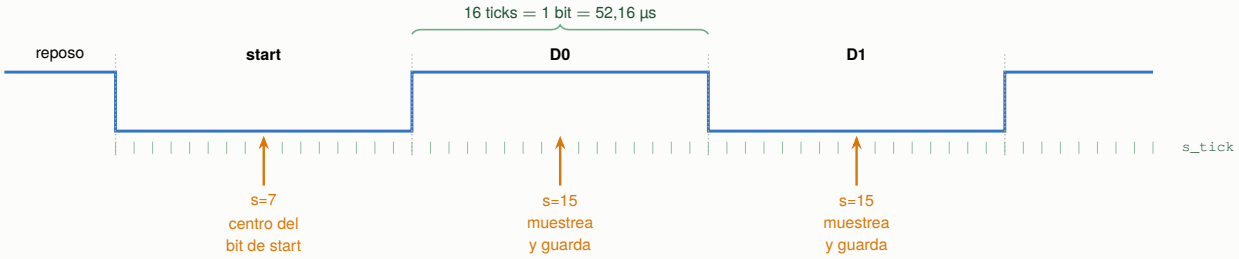
- `s_reg`: cuenta **ticks** dentro del bit actual (0 a 15).
- `n_reg`: cuenta **bits de datos** ya recibidos (0 a 7).



Los dos números mágicos son 7 y 15. En `START` se esperan **8 ticks** (contar hasta 7 desde 0) porque 8 es *medio bit*: eso deja al receptor parado en el centro del bit de start. A partir de ahí, en `DATA` se esperan **16 ticks** (contar hasta 15) por cada bit, que es *un bit entero*. Resultado: cada muestreo cae en el centro del bit correspondiente.

Dónde cae cada muestra

Esto es lo que está pasando en el cable mientras la FSM cuenta:



La cuenta exacta, en ticks desde el flanco de start

Qué	Tick nº	En bits	Estado
Detecta `rx==0`	0	0,0	`IDLE` → `START`
Centro del bit de start	8	0,5	`START` → `DATA`
Muestrea D0	24	1,5	`DATA`
Muestrea D1	40	2,5	`DATA`
… un bit cada 16 ticks …			
Muestrea D7	136	8,5	`DATA` → `STOP`
Fin del bit de stop	152	9,5	`STOP` → `IDLE`, `rx_done_tick=1`

Detalle fino: el flanco de start no está alineado con el tick

El `baud_gen` corre libre, así que cuando llega el flanco de bajada de `rx` el contador puede estar en cualquier valor entre 0 y 325. Es decir: el primer tick después de entrar en `START` llega en un momento aleatorio, entre 0 y 3,26 µs después.

Eso mete un error de hasta **un tick = 1/16 de bit = 6,25 por ciento** en la ubicación del punto

de muestreo. Sumado al 0,147 por ciento de deriva acumulada, el peor caso queda en ∼8 por ciento de bit de corrimiento respecto del centro ideal — todavía muy lejos del 50 por ciento que sería un error. **Este es el motivo real por el que se sobremuestrea 16 veces y no 4:** con 4 ticks por bit, la incertidumbre sería de 25 por ciento y el margen se pondría incómodo.

4.5 Paso 4 — `rx_done_tick`: el aviso de un ciclo

Cuando la FSM termina el bit de stop levanta `rx_done_tick` durante **un ciclo de clock**. No es un flag que queda prendido: es un pulso.

Por qué un pulso y no un flag

Un flag habría que apagarlo, y apagarlo requiere que alguien avise que ya lo leyó — lo que trae un ida y vuelta de handshake. Un pulso de un ciclo se autolimpia: quien lo tiene que ver lo ve, porque todos miran el mismo clock. Acá el pulso va directo a la entrada `wr` de la FIFO, así que el efecto es "escribí este byte, ahora".

4.6 Paso 5 — `fifo_rx`: la frontera entre los dos mundos

Este es el punto exacto donde el byte cambia de velocidad.

Qué pasa, ciclo por ciclo

Ciclo N	`uart_rx` está en `STOP` con `s_reg==15` y llega `s_tick`. Combinacionalmente: `rx_done_tick = 1`.
Flanco $N \to N+1$	La FIFO escribe el byte en `array_reg[w_ptr]`, avanza `w_ptr` y baja `empty_reg`.
Ciclo $N+1$	`rx_empty == 0`. El `intf` lo ve en el mismo ciclo, porque es combinacional sobre `empty_reg`. Y `r_data` ya muestra el byte, sin esperar.

La FIFO es *first-word fall-through*: `assign r_data = array_reg[r_ptr_reg];` es una lectura combinacional. No hay que pedir el dato y esperar un ciclo: el dato de la cabeza de la cola **siempre** está en la salida.

El truco de los dos flags

`w_ptr` y `r_ptr` tienen el mismo ancho (4 bits), así que cuando son iguales la cola puede estar **vacía o llena** — son el mismo estado de los punteros. Por eso el módulo lleva dos registros aparte, `full_reg` y `empty_reg`, en vez de deducirlo comparando punteros. El `case ({wr, rd})` es lo que los mantiene: escribir baja `empty`, leer baja `full`, y solo si el puntero que avanza alcanza al otro se levanta el flag correspondiente.

¿Y para qué sirve la cola, si el `intf` es muchísimo más rápido?

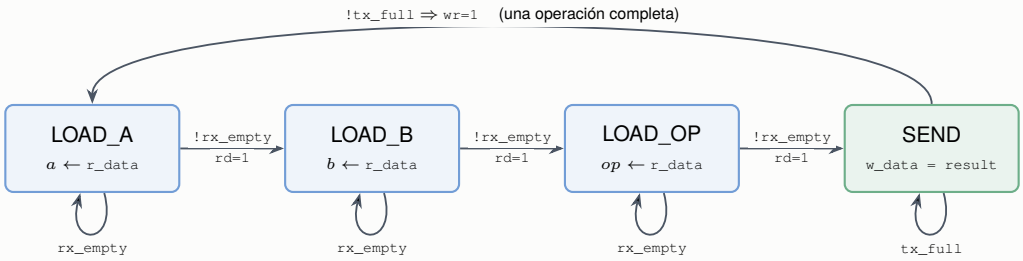
Dos razones concretas.

Una: el `intf` podría estar ocupado en `SEND` esperando a que se libere la FIFO de transmisión justo cuando llega un byte nuevo. Sin cola, ese byte se perdería (eso se llama *overrun*).

Dos: la cola es lo que permite que la PC dispare las tres tramas seguidas sin esperar respuesta. Llegan tres bytes en 1,56 ms y quedan guardados; el `intf` los va sacando a su ritmo. Con 16 lugares de profundidad hay margen de sobra.

4.7 Paso 6 — `intf`: armar la operación

La FSM del `intf` vive en el mundo rápido: un estado por ciclo de clock. Su trabajo es trivial en comparación con lo que tardó el byte en llegar.



los autoloops son la espera: aquí la FSM pasa el 99,99 por ciento del tiempo

Los tres estados de carga son idénticos

Cada uno hace exactamente lo mismo: mirar `rx_empty`, y si hay algo, levantar `rd` y copiar `r_data` a su registro. Lo único que cambia es *en qué registro* guarda y *a qué estado* va. Los autoloops (quedarse en el mismo estado mientras `rx_empty` está en 1) son la espera: la FSM se queda ahí, ciclo tras ciclo, sin hacer nada, durante los ∼52 000 ciclos que tarda en llegar el byte siguiente.

De `LOAD_OP` solo se toman los 6 bits bajos: `op_next = r_data[NB_OP-1:0];` Los dos bits altos del byte se descartan.

4.8 Paso 7 — La ALU: el bloque sin reloj

Cuando el `intf` pasa de `LOAD_OP` a `SEND`, en ese mismo flanco se cargan `op_reg` y ya estaban cargados `a_reg` y `b_reg`. Como la ALU es combinacional, su salida se actualiza **sola**, sin ningún flanco de clock, apenas cambian sus entradas.

La ALU como una calculadora siempre encendida

No hay que "disparar" la ALU ni esperar que termine. Es un enredo de compuertas: poneles A, B y OP en la entrada y unos nanosegundos después el resultado aparece en la salida. Mientras A, B y OP no cambien, el resultado se queda ahí quieto para siempre.

Por eso el `intf` no necesita ningún estado de "calcular". Cuando llega a `SEND`, el resultado *ya está*: `assign w_data = i_alu_result;` conecta la salida de la ALU directo a la entrada de la FIFO de transmisión.

Lo único que la ALU le exige al resto

Que su retardo combinacional entre A/B/OP y el resultado entre en **un período de clock (10 ns)**, porque ese resultado se captura en el flip-flop de la FIFO en el flanco siguiente. Para una ALU de 8 bits en un Artix-7 esto sobra: el camino crítico es el sumador con acarreo, del

orden de 2 a 3 ns.

Si la ALU fuera de 64 bits o hiciera una multiplicación, este camino podría no cerrar timing y habría que agregar un estado de espera o registrar la salida en el medio (*pipeline*).

5 El viaje de vuelta: de la ALU a la PC

5.1 Paso 8 — `SEND` y `fifo_tx`

En `SEND`, si `tx_full` está en 0, el `intf` levanta `wr` durante un ciclo. En el flanco, la FIFO de transmisión guarda el resultado y baja su `empty`. Aprovecha el mismo ciclo para enganchar el carry en un registro, que es lo que después se ve en el LED 8.

5.2 Paso 9 — `uart_tx` arranca solo

Acá hay un detalle lindo del diseño. El transmisor no recibe una orden de nadie: mira si la cola tiene algo.

El arranque automático

```
assign tx_fifo_not_empty = ~tx_empty;
uart_tx u_uart_tx ( .tx_start(tx_fifo_not_empty), ... );
```

Como `uart_tx` en `IDLE` arranca apenas ve `tx_start==1`, poner ahí directamente "la cola no está vacía" hace que el transmisor se dispare solo y encadene bytes sin que nadie lo coordine. Y del otro lado:

```
.rd(tx_done_tick)
```

cuando el transmisor termina de sacar un byte, ese mismo pulso lo saca de la cola. El lazo se cierra sin una sola línea de control extra.

5.3 Paso 10 — El byte sale bit por bit

El `uart_tx` recorre los mismos cuatro estados que el receptor, pero al revés: en vez de muestrear, *maneja* la línea. En `START` la pone en 0 y espera 16 ticks; en `DATA` saca `b_reg[0]` y después de 16 ticks desplaza (`b_next = b_reg >> 1`); en `STOP` la pone en 1 durante 16 ticks más y levanta `tx_done_tick`.

El transmisor no se alinea con el tick al arrancar

Cuando `uart_tx` pasa de `IDLE` a `START`, pone la línea en 0 **inmediatamente**, en ese ciclo de clock — y recién después empieza a contar los 16 ticks. Como el `baud_gen` corre libre, el primer tick puede llegar entre 1 y 326 ciclos después. Resultado: el **bit de start puede salir hasta 3,26 µs más corto** que los demás (un 6 por ciento).

→ ¿Rompe algo? No, porque el receptor del otro lado muestrea en el *centro* de cada bit y tiene medio bit de margen. Pero es exactamente el tipo de detalle que vale la pena saber contestar si lo preguntan. Si se quisiera evitar, bastaría con condicionar la salida de `IDLE` a `if (tx_start`

`&& s_tick)`, a costa de hasta 3,26 µs de latencia extra.

El hueco entre tramas

Cuando `tx_done_tick` se levanta, en el mismo flanco pasan dos cosas: la FSM vuelve a `IDLE` y la FIFO saca el byte. Si quedó otro byte encolado, `tx_empty` sigue en 0 y el transmisor arranca de nuevo al ciclo siguiente. Es decir: entre dos tramas consecutivas hay **un ciclo de clock de hueco, 10 ns**, contra 521 600 ns de trama. Despreciable, y además la línea queda en 1 (reposo), que es exactamente lo que espera el receptor.

6 El cronograma completo

Poner los tiempos juntos es lo que hace evidente quién manda acá.

Una operación de punta a punta

Etapa	Duración	En ciclos	Quién trabaja
Trama de A	521,6 µs	52 160	cable + `uart_rx`
Trama de B	521,6 µs	52 160	cable + `uart_rx`
Trama de OP	521,6 µs	52 160	cable + `uart_rx`
Trabajo del `intf`	**40 ns**	**4**	la FSM completa
Cálculo de la ALU	∼3 ns	< 1	combinacional
Trama del resultado	521,6 µs	52 160	`uart_tx` + cable
Total	**2,086 ms**	208 644	

El 99,998 por ciento del tiempo se lo lleva el cable. La lógica que escribimos — la FSM, la ALU, las FIFOs — ocupa 4 ciclos de clock de los 208 644 que dura la operación. Todo el diseño del UART existe para adaptar un chip absurdamente rápido a un cable absurdamente lento, sin perder un solo bit en el camino.

Dónde se va todo ese tiempo

Si el sistema fuera una oficina: el cartero (el cable) tarda medio milisegundo en traer cada sobre, y el empleado (la FPGA) resuelve el trámite en 40 nanosegundos. El empleado pasa el 99,998 por ciento de su jornada mirando la puerta. Los autoloops de las FSM — `rx_empty` en `LOAD_A`, `s_tick==0` en `DATA` — son literalmente eso: mirar la puerta.

7 Tabla integradora: todas las señales de un vistazo

Quién genera qué, y quién la consume

Señal	La genera	La usa	Qué significa
`clk`	la placa	todos	el único reloj del diseño
`s_tick`	baud_gen	rx, tx	pulso de 1 ciclo, 16 por bit
`rx_sync`	el top	uart_rx	la línea serie ya sincronizada
`rx_done_tick`	uart_rx	fifo_rx (wr)	"terminé un byte"
`dout`	uart_rx	fifo_rx	el byte recién armado
`rx_empty`	fifo_rx	intf	"no hay nada para leer"
`r_data`	fifo_rx	intf	cabeza de la cola (combinacional)
`rd`	intf	fifo_rx	"sacame este byte"
`a, b, op`	intf	ALU	operandos registrados
`result, carry`	ALU	fifo_tx, LED	salida combinacional
`wr`	intf	fifo_tx	"guardame el resultado"
`tx_full`	fifo_tx	intf	"no entra nada más"
`tx_start`	fifo_tx	uart_tx	en realidad es `~tx_empty`
`tx_done_tick`	uart_tx	fifo_tx (rd)	"salí un byte entero"

8 Las siete preguntas que más se hacen

1. **¿Por qué 16 ticks por bit y no 1?**
   Con 1 tick por bit no hay forma de encontrar el centro: se muestrearía en un punto arbitrario, tal vez justo en la transición. Con 16, el punto de muestreo queda determinado con precisión de 1/16 de bit. Es el estándar de facto en UARTs por hardware.

2. **¿Por qué `s_reg` cuenta hasta 7 en `START` y hasta 15 en `DATA`?**
   7 (o sea 8 ticks) es *medio* bit: lleva al centro del start. 15 (16 ticks) es *un* bit entero: mantiene el centrado en cada bit siguiente. Media más entera, entera, entera…

3. **¿El `s_tick` es un reloj?**
   No. Es una *habilitación*. Todos los flip-flops del diseño se disparan con `clk`; el tick aparece como una condición `if (s_tick)` adentro de la lógica combinacional. Hay un único dominio de reloj.

4. **¿Por qué hace falta la FIFO si el `intf` es 52 000 veces más rápido que el cable?**
   Porque "más rápido" no es "siempre disponible". Si el `intf` está trabado en `SEND` justo cuando termina de llegar un byte, sin cola ese byte se pierde. La FIFO desacopla *disponibilidad*, no solo velocidad.

5. **¿Cuándo calcula la ALU?**
   Siempre. No tiene reloj ni señal de arranque. Su salida vale lo que valgan sus entradas, perma-

nentemente. El `intf` no la "dispara": simplemente se asegura de que A, B y OP estén cargados antes de leer el resultado.

6. **¿Qué pasa si la PC manda 4 bytes en vez de 3?**
   El cuarto queda en la FIFO y se toma como el *A* de la operación siguiente. El protocolo no tiene forma de resincronizarse: es posicional. Si se quisiera robustez habría que agregar un byte de comando o un *timeout* que vuelva a `LOAD_A`.

7. **¿Por qué todos los `case` tienen `default`?**
   Por dos motivos distintos. En la lógica combinacional, para que ninguna señal quede sin asignar en algún camino — eso sintetizaría un *latch*. Y en las FSM, porque es la *máquina de estado segura* de la teórica: ante un estado inválido (un glitch, una partícula), el ciclo siguiente vuelve al estado inicial en vez de quedar colgada.

9 Autoevaluación

Si podés contestar estas doce sin volver atrás, el flujo está entendido.

1. ¿Cuántos ciclos de clock dura un bit en el cable? ¿Y cuántos ticks?
2. ¿Quién decide que empieza una trama, y cómo se entera?
3. Si cambiaras el baud rate a 115200, ¿qué número tocarías y cuánto valdría?
4. ¿Por qué el shift register del receptor mete el bit nuevo por izquierda?
5. ¿Qué pasaría si en `START` se contara hasta 15 en vez de hasta 7?
6. Un byte llega y `rx_empty` baja. ¿En qué ciclo lo puede leer el `intf`, y por qué no antes?
7. ¿Por qué `r_data` está disponible sin esperar un ciclo después de `rd`?
8. ¿Cuántos ciclos de clock tarda el `intf` en procesar los tres bytes, una vez que ya llegaron todos?
9. ¿Por qué la ALU no necesita una señal de "empezar" ni una de "listo"?
10. ¿Quién le dice al `uart_tx` que tiene que transmitir?
11. ¿Cuánto tiempo de la operación completa es lógica nuestra y cuánto es cable? Dalo en porcentaje.
12. ¿Por qué la línea `i_rx` pasa por dos flip-flops antes de entrar al receptor, y qué le cuesta eso al diseño?